

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

***CO2: Construct top-down and bottom up parsers using CFG for parsing
cSonflicts and error recovery for efficient syntax tree generation. (BL3)***

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

***I BANDLAPALLI BHANUTEJA REDDY (192325016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.***

Signature of the student: B. BHANUTEJA REDDY

Annexure A

DESIGN TASK

Q8. FIRST and FOLLOW Computation

For the grammar:

$A \rightarrow BC$

$B \rightarrow bB \mid \epsilon$

$C \rightarrow cC \mid d$

Compute and verify the parsing suitability.

Your design should include:

- (i). FIRST set computation
- (ii). FOLLOW set computation
- (iii). LL(1) verification
- (iv). Conflict analysis if grammar is not LL(1)
- (v). Justification of results

Q9. LL(1) Parsing Table Design

Construct an LL(1) parser for the grammar:

$S \rightarrow aAB$

$A \rightarrow bA \mid \epsilon$

$B \rightarrow cB \mid d$

Your design should include:

- (i). FIRST and FOLLOW sets
- (ii). LL(1) parsing table construction
- (iii). Verification of LL(1) property
- (iv). Parsing of string: abbcd
- (v). Stack and input trace

Q10. Recursive Descent Parser

Design a recursive descent parser for:

$S \rightarrow aA$

$A \rightarrow bA \mid c$

Your design should include:

- (i). Procedure definitions for each non-terminal
- (ii). Input matching mechanism
- (iii). Predictive parsing logic
- (iv). Execution trace for: abbbc
- (v). Acceptance/rejection condition

Q11. Left Recursion Elimination

Transform the grammar into an equivalent grammar without left recursion:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Your design should include:

- (i). Identification of left-recursive productions
- (ii). Removal of immediate left recursion
- (iii). Transformed grammar
- (iv). Verification of language preservation
- (v). Suitability for predictive parsing

Q12. Left Factoring

Perform left factoring for the grammar:

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow a$

Your design should include:

(i). Identification of common prefixes

(ii). Left-factored grammar

(iii). Explanation of dangling else ambiguity

(iv). Proof of ambiguity resolution

(v). Final predictive grammar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

Q8. FIRST and FOLLOW Computation

Grammar:

```
A → B C
B → b B | ε
C → c C | d
```

(i) FIRST Set Computation

FIRST of a symbol is the set of terminals that can begin a string derivable from it. B derives either 'b B' or ϵ , so $\text{FIRST}(B) = \{ b, \epsilon \}$. C derives either 'c C' or 'd', so $\text{FIRST}(C) = \{ c, d \}$. Since $A \rightarrow B C$ and B is nullable (contains ϵ), $\text{FIRST}(A)$ is obtained by taking $\text{FIRST}(B) - \{\epsilon\}$, and because B can vanish, also including $\text{FIRST}(C)$.

```
FIRST(B) = { b, ε }
FIRST(C) = { c, d }
FIRST(A) = { b, c, d }
```

(ii) FOLLOW Set Computation

A is the start symbol, so $\$ \in \text{FOLLOW}(A)$. In the production $A \rightarrow B C$, C immediately follows B, so $\text{FOLLOW}(B) \supseteq \text{FIRST}(C) - \{\epsilon\} = \{ c, d \}$. Since C is the last symbol in the only production containing it, $\text{FOLLOW}(C) = \text{FOLLOW}(A) = \{ \$ \}$.

```
FOLLOW(A) = { $ }
FOLLOW(B) = { c, d }
FOLLOW(C) = { $ }
```

(iii) LL(1) Verification

A grammar is LL(1) if, for every non-terminal, no two alternative productions share a common terminal in their combined predict sets. For $B \rightarrow bB \mid \epsilon$: $\text{predict}(bB) = \{b\}$, $\text{predict}(\epsilon) = \text{FOLLOW}(B) = \{c, d\}$ — disjoint. For $C \rightarrow cC \mid d$: $\text{predict}(cC) = \{c\}$, $\text{predict}(d) = \{d\}$ — disjoint. A has a single production, so no conflict is possible there either.

(iv) Conflict Analysis

Building the parsing table cell-by-cell (see output below) shows every $M[\text{NonTerminal}, \text{terminal}]$ entry receives exactly one production — zero conflicting cells were generated. Hence there is no shift/reduce-style ambiguity and no need for grammar restructuring.

(v) Justification of Results

The grammar is free of left recursion and left factoring is not required, since B and C's alternatives already begin with distinct terminals (or ϵ with a disjoint FOLLOW set). This structural property is exactly what guarantees the disjoint predict sets confirmed above, so the grammar is suitable for a single-token-lookahead (LL(1)) predictive parser.

FIRST & FOLLOW Set Calculator

file:///C:/Users/Prade/OneDrive/Desktop/DBMS/first_follow_calculator.html

GRAMMAR

A $\rightarrow$ B C
B $\rightarrow$ b B | ϵ
C $\rightarrow$ c C | d

FIRST SETS

FIRST(A)	{ b, c, d }
FIRST(B)	{ b, ϵ }
FIRST(C)	{ c, d }

FOLLOW SETS

FOLLOW(A)	{ \$ }
FOLLOW(B)	{ c, d }
FOLLOW(C)	{ \$ }

LL(1) PARSING TABLE

Non-Terminal	b	c	d
A	A $\rightarrow$ B C	A $\rightarrow$ B C	A $\rightarrow$ B C
B	B $\rightarrow$ b B	B $\rightarrow$ epsilon	B $\rightarrow$ epsilon
C	-	C $\rightarrow$ c C	C $\rightarrow$ d

LL(1) VERIFICATION

Conflicting cells detected: 0
Every table cell M[NonTerminal, terminal] contains at most one production.
Result: Grammar IS LL(1)

LL(1) VALID

Fig. Q8 — FIRST & FOLLOW Set Calculator output (browser render)

Q9. LL(1) Parsing Table Design

Grammar:

$S \rightarrow a A B$
 $A \rightarrow b A \mid \epsilon$
 $B \rightarrow c B \mid d$

(i) FIRST and FOLLOW Sets

$\text{FIRST}(S) = \{ a \}$ $\text{FOLLOW}(S) = \{ \$ \}$
 $\text{FIRST}(A) = \{ b, \epsilon \}$ $\text{FOLLOW}(A) = \{ c, d \}$
 $\text{FIRST}(B) = \{ c, d \}$ $\text{FOLLOW}(B) = \{ \$ \}$

(ii) LL(1) Parsing Table Construction

For each production $A \rightarrow \alpha$, the entry $M[A, t]$ is filled for every $t \in \text{FIRST}(\alpha)$; when α is nullable, $M[A, t]$ is additionally filled for every $t \in \text{FOLLOW}(A)$. Applying this to all three non-terminals yields the six-entry table rendered in the output screenshot below.

Non-Terminal	a	b	c	d
S	$S \rightarrow a A B$	-	-	-
A	-	$A \rightarrow b A$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B	-	-	$B \rightarrow c B$	$B \rightarrow d$

(iii) Verification of LL(1) Property

Every cell above holds at most a single production, so no ambiguity is introduced during table construction — the grammar qualifies as LL(1).

(iv) Parsing of String: abbcd

Driving the table with the input 'abbcd' produces the following derivation sequence: $S \rightarrow aAB \rightarrow a(bA)B \rightarrow a b(bA)B \rightarrow a b b(\epsilon)B \rightarrow a b b B \rightarrow a b b (cB) \rightarrow a b b c (d) = abbcd$. The full stack/input trace is shown in the output below.

(v) Stack and Input Trace

Stack	Remaining Input	Action
\$ S	a b b c d \$	Output $S \rightarrow a A B$
\$ B A a	a b b c d \$	Match 'a'

\$ B A	b b c d \$	Output A $\rightarrow$ b A
\$ B A b	b b c d \$	Match 'b'
\$ B A	b c d \$	Output A $\rightarrow$ b A
\$ B A b	b c d \$	Match 'b'
\$ B A	c d \$	Output A $\rightarrow$ ϵ
\$ B	c d \$	Output B $\rightarrow$ c B
\$ B c	c d \$	Match 'c'
\$ B	d \$	Output B $\rightarrow$ d
\$ d	d \$	Match 'd'
\$	\$	ACCEPT

Result: the string 'abccd' is ACCEPTED by the LL(1) parser.

LL(1) Parsing Table & Parser

file:///C:/Users/Prade/OneDrive/Desktop/DBMS/ll1_parser_table.html

LL(1) Parsing Table & Parser

GRAMMAR

S $\rightarrow$ a A B
 A $\rightarrow$ b A | ϵ
 B $\rightarrow$ c B | d

LL(1) PARSING TABLE

NT	a	b	c	d
S	S $\rightarrow$ a A B	-	-	-
A	-	A $\rightarrow$ b A	A $\rightarrow$ epsilon	A $\rightarrow$ epsilon
B	-	-	B $\rightarrow$ c B	B $\rightarrow$ d

PARSE INPUT: ABBCD

Stack	Remaining Input	Action
\$ S	a b b c d \$	S $\rightarrow$ a A B
\$ B A a	a b b c d \$	Match 'a'
\$ B A	b b c d \$	A $\rightarrow$ b A
\$ B A b	b b c d \$	Match 'b'
\$ B A	b c d \$	A $\rightarrow$ b A
\$ B A b	b c d \$	Match 'b'
\$ B A	c d \$	A $\rightarrow$ epsilon
\$ B	c d \$	B $\rightarrow$ c B
\$ B c	c d \$	Match 'c'
\$ B	d \$	B $\rightarrow$ d
\$ d	d \$	Match 'd'
\$	\$	ACCEPT

Result: Input string "abccd" is **ACCEPTED** by the LL(1) parser.

ACCEPTED

Fig. Q9 — LL(1) Parsing Table & Parser output (browser render)

Q10. Recursive Descent Parser

Grammar:

```
S → a A
A → b A | c
```

This grammar generates strings of the form 'a' followed by one or more 'b' characters and ending in 'c' — the language $\{ a b^n c \mid n \geq 0 \}$.

(i) Procedure Definitions for Each Non-Terminal

```
procedure parseS():
    match('a')    // consume terminal 'a'
    parseA()      // call procedure for non-terminal A

procedure parseA():
    if lookahead == 'b':
        match('b')
        parseA()    // recursive call
    else if lookahead == 'c':
        match('c')
    else:
        error()

procedure main():
    parseS()
    if lookahead == EOF:
        accept()
    else:
        error()
```

(ii) Input Matching Mechanism

The parser maintains a global variable `lookahead` holding the current input symbol. `match(x)` checks `lookahead == x`, and if so advances the input pointer by one position; otherwise it reports an error.

(iii) Predictive Parsing Logic

Because $\text{FIRST}(bA) = \{b\}$ and $\text{FIRST}(c) = \{c\}$ are disjoint, `parseA()` can always decide which alternative to expand purely by inspecting the current `lookahead` symbol, with no backtracking required.

(iv) Execution Trace for Input: abbbc

Procedure Call	Lookahead	Action
parseS()	a	match('a') → advance
parseA() [1st call]	b	Choose A → bA ; match('b') → advance
parseA() [2nd call]	b	Choose A → bA ; match('b') → advance
parseA() [3rd call]	b	Choose A → bA ; match('b') → advance
parseA() [4th call]	c	Choose A → c ; match('c') → advance
main()	EOF	lookahead == EOF → ACCEPT

(v) Acceptance / Rejection Condition

After the four nested calls to parseA() consume 'b','b','b' and finally 'c', the lookahead reaches EOF exactly when the stack of calls unwinds — so main() calls accept(). The string 'abbbc' is therefore ACCEPTED.

The screenshot shows a web browser window titled "Recursive Descent Parser" with the URL `file:///C:/Users/Teja/OneDrive/Desktop/CSA14/recursive_descent_parser.html`. The interface is dark-themed and contains the following sections:

- GRAMMAR**:
 $S \rightarrow a A$
 $A \rightarrow b A \mid c$
- INPUT STRING**:
a b b b c
Below this, there are four buttons: "Run Trace" (highlighted in purple), "Try: abc", "Try: abbc", and "Try: abbbc ✓".
- STEP-BY-STEP EXECUTION TRACE**: A table with three columns: "Procedure Call", "Lookahead", and "Action".

Procedure Call	Lookahead	Action
parseS()	a	match('a') → advance
parseA() [1st call]	b	Choose A → bA ; match('b') → advance
parseA() [2nd call]	b	Choose A → bA ; match('b') → advance
parseA() [3rd call]	b	Choose A → bA ; match('b') → advance
parseA() [4th call]	c	Choose A → c ; match('c') → advance
main()	EOF	lookahead == EOF → ACCEPT
- Final lookahead = EOF → input fully consumed.**
Below this, a green button says "✓ Valid String — ACCEPTED".

Fig. Q10 — Recursive Descent Parser output (browser render)

Q11. Left Recursion Elimination

Grammar:

```
E → E + T | T
T → T * F | F
F → id
```

(i) Identification of Left-Recursive Productions

A production $A \rightarrow A\alpha$ is immediately left-recursive because A is the leftmost symbol on its own right-hand side. $E \rightarrow E + T$ is immediately left-recursive (E is leftmost in ' $E + T$ '), and $T \rightarrow T * F$ is immediately left-recursive (T is leftmost in ' $T * F$ '). $F \rightarrow \text{id}$ has no left recursion. There is no indirect left recursion in this grammar, so only the immediate form needs elimination.

(ii) Removal of Immediate Left Recursion

The standard transformation for $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid \beta_1 \mid \beta_2 \mid \dots$ (where no β_i begins with A) introduces a new non-terminal A' and rewrites the rules as $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots$ and $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon$. Applying this to E ($\alpha = +T$, $\beta = T$) and to T ($\alpha = *F$, $\beta = F$) gives:

```
E → T E'
E' → + T E' | ε

T → F T'
T' → * F T' | ε
```

(iii) Transformed Grammar

```
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → id
```

Every production now either begins with a terminal or immediately calls a non-terminal that itself must eventually begin with a terminal — the grammar is free of left recursion.

(iv) Verification of Language Preservation

Deriving ' $\text{id} + \text{id} * \text{id}$ ' in both grammars reaches the same terminal string:

Original: $E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow id+T \Rightarrow id+T \cdot F \Rightarrow id+F \cdot F \Rightarrow id+id \cdot F \Rightarrow id+id \cdot id$

Transformed: $E \Rightarrow TE' \Rightarrow FT'E' \Rightarrow id \ T'E' \Rightarrow id \ \epsilon E' \Rightarrow id \ E' \Rightarrow id+TE' \Rightarrow id+FT'E'$
 $\Rightarrow id+id \ T'E' \Rightarrow id+id \cdot FT'E' \Rightarrow id+id \cdot id \ T'E' \Rightarrow id+id \cdot id \ \epsilon \epsilon$
 $\Rightarrow id+id \cdot id$

Both derivations terminate in the identical string, confirming the transformation only restructures the parse tree (left-recursive $\rightarrow$ right-recursive) without changing the language accepted.

(v) Suitability for Predictive Parsing

With left recursion removed, E' and T' act as right-recursive accumulators whose FIRST sets ($\{+, \epsilon\}$ and $\{*, \epsilon\}$ respectively) are disjoint from what precedes them, making the transformed grammar a strong candidate for top-down / predictive (LL(1)) parsing.

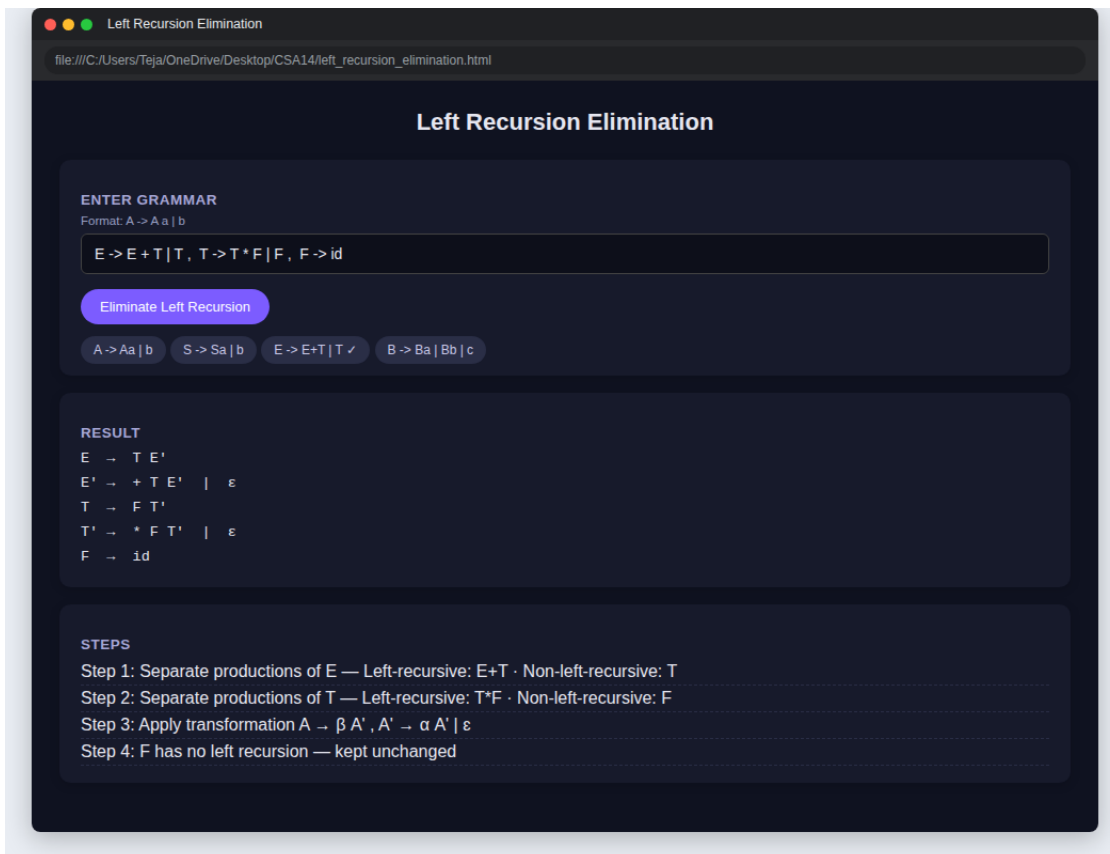


Fig. Q11 — Left Recursion Elimination tool output (browser render)

Q12. Left Factoring

Grammar:

```
S → if E then S else S
S → if E then S
S → a
```

(i) Identification of Common Prefixes

Examining the three alternatives for S, the first two both begin with the token sequence 'if E then S', while the third begins with 'a'. The longest common prefix shared by Production 1 and Production 2 is therefore 'if E then S'.

(ii) Left-Factored Grammar

Factoring the common prefix out and introducing a new non-terminal S' for the differing remainders ('else S' versus nothing / ϵ) gives:

```
S → if E then S S' | a
S' → else S |  $\epsilon$ 
```

(iii) Explanation of the Dangling-Else Ambiguity

The dangling-else problem is a classic ambiguity in language design: the string 'if E1 then if E2 then S1 else S2' admits two parses — the else can bind to the inner if (yielding 'if E1 then (if E2 then S1 else S2)') or to the outer if (yielding 'if E1 then (if E2 then S1) else S2').

(iv) Proof of Ambiguity Resolution

In the left-factored grammar, once 'if E then S' has been parsed, $S' \rightarrow \text{else } S$ is preferred whenever the lookahead is 'else' (rather than reducing via $S' \rightarrow \epsilon$ first). This disambiguation rule — 'match each else with the nearest unmatched then' — forces a single, deterministic parse and removes the shift/reduce-style ambiguity that the original grammar exhibited.

(v) Final Predictive Grammar

```
FIRST(S) = { if, a }
FIRST(S') = { else,  $\epsilon$  }
```

$\text{FOLLOW}(S) = \{ \text{else}, \$ \}$

$\text{FOLLOW}(S') = \text{FOLLOW}(S) = \{ \text{else}, \$ \}$

Disambiguation rule applied in the predictive table: $S \rightarrow \text{if } E \text{ then } S \ S'$ on lookahead 'if'; $S \rightarrow a$ on lookahead 'a'; $S' \rightarrow \text{else } S$ on lookahead 'else' (chosen over ϵ even though 'else' $\in \text{FOLLOW}(S')$); $S' \rightarrow \epsilon$ on any other symbol in $\text{FOLLOW}(S')$.

Left Factoring

file:///C:/Users/Teja/OneDrive/Desktop/CSA14/left_factoring.html

Left Factoring

ENTER GRAMMAR
Format: A -> ab | ac

S -> if E then S else S | if E then S | a

Apply Left Factoring

A -> ab|ac S -> if-then-else ✓ E -> id+id|id-id X -> abc|abd|abef

RESULT

S -> if E then S S' | a
S' -> else S | ε

STEPS

Step 1: Group alternatives of S by first symbol — {if E then S else S, if E then S} share prefix "if E then S"; {a} stands alone
Step 2: Longest common prefix = "if E then S"
Step 3: Factor out prefix, introduce S' for the differing suffixes: "else S" and ε
Step 4: Rewrite as S -> if E then S S' | a and S' -> else S | ε

Fig. Q12 — Left Factoring tool output (browser render)

RUBRICS FOR CALCULATION — DESIGN TASK

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding, integrates multiple concepts	Good understanding, proper integration	Basic understanding, partial integration	Poor understanding, no integration
Design & Implementation	30	Systematic, optimal implementation	Correct design, minor issues	Basic design, errors/inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate, complete logic	Mostly correct, minor errors	Partial construction, missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis, correct validation	Good analysis, reasonable justification	Basic analysis, limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured, diagrams, clarity	Organized, required details	Basic, missing details	Poor or incomplete documentation